

Skill 11

Task 1

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>
int main() {
    char command[100];
    int status;
    printf("Enter a command: ");
    fgets(command, sizeof(command), stdin);
    // Remove newline
    command[strcspn(command, "\n")] = '\0';
    // Check empty command
    if (strlen(command) == 0) {
        printf("ERROR: Empty command.\n");
        return 1;
    }
    printf("\nExecuting: %s\n", command);
    pid_t pid = fork();
    // Check fork error
    if (pid < 0) {
        perror("ERROR: fork failed");
        return 1;
    }
    // Child process
    if (pid == 0) {
        execl("/bin/sh", "sh", "-c", command, NULL);
        // If exec fails
    }
```

```
}
// Parent process
else {
    // Wait for child
    if (waitpid(pid, &status, 0) == -1) {
        perror("ERROR: waitpid failed");
        return 1;
    }
    // Check execution result
    if (WIFEXITED(status)) {
        int exit_status = WEXITSTATUS(status);
        if (exit_status == 0) {
            printf("\nSUCCESS: Command executed correctly.\n");
        } else {
            printf("\nERROR: Command failed.\n");
            printf("Exit status: %d\n", exit_status);
            // Log failure
            FILE *log = fopen("error.log", "a");
            if (log != NULL) {
                fprintf(log,
                    "Command failed: %s | Exit status: %d\n",
                    command,
                    exit_status);
                fclose(log);
                printf("Failure logged in error.log\n");
            } else {
                perror("ERROR: Cannot open log file");
            }
        }
    }
}
```

```

        // Check if process was terminated by signal
    else if (WIFSIGNALED(status)) {
        printf("\nERROR: Process terminated by signal.\n");
        printf("Signal number: %d\n",
            WTERMSIG(status));
        // Log signal failure
        FILE *log = fopen("error.log", "a");
        if (log != NULL) {
            fprintf(log,
                "Command terminated by signal: %s | Signal: %d\n",
                command,
                WTERMSIG(status));
            fclose(log);
        }
    }
}
printf("\nProgram recovered successfully.\n");
printf("You can enter another command.\n");
return 0;
}

```

Output:

```

root@Tejasri:~# gcc error_handling.c -o error_handling
root@Tejasri:~# ./error_handling
Enter a command: ls

Executing: ls
Commands      Task.save      copy            input_redirection  race_condition.c  shell_signal.c
Commands.c    Task1          copy.c          input_redirection.c  redirect          shopping
Dashboard     Task1.c        cow             jobs               redirect.c        shopping.c
Dashboard.c   Task1.c.save  cow.c          jobs.c            redirection       shopping.c.save
File1.tx      Task2         deadlock        jobtable          redirection.c     signal_demo
LICENSE       Task2.c       deadlock.c     jobtable.c        redirection.c     signal_demo.c
LICENSE       Task2.c.save  destination.txt links.c            result.txt        signals
Makefile      Task3         directory      map               resume            signals.c
Memory        Task3.c       directory.c    map.c             resume.c          sigstp_demo
Memory.c      Task4.c       docs           mutex_counter     sTask             sigstp_demo.c
MyShell       Week3.c.save  error.txt      mutex_counter.c   sTask.c           skill
MyShell.c     a.out         error_handling  navigation         sTask1            skill.c
OSSP          append_redirect  error_handling.c  navigation.c      sTask1.c          source.txt
Ptask1        append_redirect.c  export          obj               sTask2            src
Ptask1.c      append_redirection  export.c        output.txt        sTask2.c          state.txt
Ptask2        append_redirection.c  filecopy       output_redirect   sTask3            stedrr_redirect
Ptask2.c      assets         filename.c     output_redirect.c  sTask3.c          stedrr_redirect.c
README.md     assets         foreground     output_redirection sTask4            stedrr_redirection
S5Task1       background     foreground.c   output_redirection.c sTask4.c          task3
S5Task1.c     background.c   gitignore     pipe              scripts            task3.c
S5Task2       bin            ground         pipe.c            server             tests
S5Task2.c     client         ground.c      posix             server.c          week1.c.save
Session3      client.c      groups        posix.c           session3           week2
Session3.c    combined      groups.c      proc              session3.c        week2.c
Shell         combined.c    include       program           shell              week5.save
Shell.c       command_pipe.c  incde.c      program.c         shell.c
Signals       complex       incode_links  race_condition    shell_signal
Signals.c     complex.c

SUCCESS: Command executed correctly.

Program recovered successfully.
You can enter another command.
root@Tejasri:~#

```

Task 2:

To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

```
GNU nano 7.2 memory.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main() {
    char *name;
    int *numbers;
    printf("Memory Management Test\n");
    printf("=====\n");
    /* Allocate memory for name */
    name = (char *)malloc(50 * sizeof(char));
    if (name == NULL) {
        printf("Error: Memory allocation failed for name.\n");
        return 1;
    }
    strcpy(name, "Valgrind Memory Test");
    printf("Name: %s\n", name);
    /* Allocate memory for integer array */
    numbers = (int *)malloc(5 * sizeof(int));
    if (numbers == NULL) {
        printf("Error: Memory allocation failed for numbers.\n");
        /* Release previously allocated memory */
        free(name);
        return 1;
    }
    /* Store values */
    for (int i = 0; i < 5; i++) {
        numbers[i] = (i + 1) * 10;
    }
    printf("Numbers: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }
}
```

```
        printf("%d ", numbers[i]);
    }
    printf("\n");
    /* Release allocated memory */
    free(name);
    name = NULL;
    free(numbers);
    numbers = NULL;
    printf("\nMemory released successfully.\n");
    printf("Cleanup completed.\n");
    return 0;
}
Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute
```

Output:

```
root@Tejasri:~# nano memory.c
root@Tejasri:~# gcc memory.c -o memory
root@Tejasri:~# ./memory
Memory Management Test
=====
Name: Valgrind Memory Test
Numbers: 10 20 30 40 50

Memory released successfully.
Cleanup completed.
root@Tejasri:~# |
```

Task 3:

To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* Module 1: Input */
int get_input(char *name, int size) {
    printf("Enter your name: ");
    if (fgets(name, size, stdin) == NULL) {
        return 0;
    }
    name[strcspn(name, "\n")] = '\0';
    if (strlen(name) == 0) {
        return 0;
    }
    return 1;
}
/* Module 2: Processing */
int process_data(char *name) {
    if (name == NULL || strlen(name) == 0) {
        return 0;
    }
    printf("\nProcessing data...\n");
    return 1;
}
/* Module 3: Output */
void display_result(char *name) {
    printf("\n===== \n");
    printf("          INTEGRATION RESULT\n");
    printf("===== \n");
    printf("Hello, %s!\n", name);
    printf("All modules executed successfully.\n");
    printf("===== \n");
}
```

```

/* Main module */
int main() {
    char name[100];
    printf("==== END-TO-END MODULE TEST =====\n");
    /* Connect Module 1 */
    if (!get_input(name, sizeof(name))) {
        printf("ERROR: Invalid input.\n");
        return 1;
    }
    /* Connect Module 2 */
    if (!process_data(name)) {
        printf("ERROR: Data processing failed.\n");
        return 1;
    }
    /* Connect Module 3 */
    display_result(name);
    printf("\nSystem execution completed successfully.\n");
    return 0;
}
```

Output:

```
root@Tejasri:~# nano integration.c
root@Tejasri:~# gcc integration.c -o integration
root@Tejasri:~# ./integration
===== END-TO-END MODULE TEST =====
Enter your name: Tejasri

Processing data...

=====
                INTEGRATION RESULT
=====
Hello, Tejasri!
All modules executed successfully.
=====

System execution completed successfully.
root@Tejasri:~#
```

Task 4:

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

```
struct TestCase {
    int a;
    int b;
    int expected;
};

/* Run addition test cases */
void test_addition() {
    struct TestCase tests[] = {
        {2, 3, 5},
        {10, 20, 30},
        {-5, 5, 0},
        {7, 8, 15}
    };
    int total = 4;
    int passed = 0;
    printf("\n===== ADDITION TESTS =====\n");
    for (int i = 0; i < total; i++) {
        int result = add(tests[i].a, tests[i].b);
        printf("Test Case %d: ", i + 1);
        if (result == tests[i].expected) {
            printf("PASS\n");
            passed++;
        } else {
            printf("FAIL\n");
        }
        printf("Expected: %d | Actual: %d\n",
            tests[i].expected, result);
    }
    printf("Passed: %d/%d\n", passed, total);
}

/* Run multiplication test cases */
```

```

int total = 4;
int passed = 0;
printf("\n===== MULTIPLICATION TESTS =====\n");
for (int i = 0; i < total; i++) {
    int result = multiply(tests[i].a, tests[i].b);
    printf("Test Case %d: ", i + 1);
    if (result == tests[i].expected) {
        printf("PASS\n");
        passed++;
    } else {
        printf("FAIL\n");
    }
    printf("Expected: %d | Actual: %d\n",
        tests[i].expected, result);
}
printf("Passed: %d/%d\n", passed, total);
}
/* Generate final report */
void generate_report() {
    printf("\n===== \n");
    printf("          TEST REPORT\n");
    printf("===== \n");
    printf("Addition Tests      : Completed\n");
    printf("Multiplication Tests : Completed\n");
    printf("Output Verification : Completed\n");
    printf("Result Comparison   : Completed\n");
    printf("Test Automation     : Completed\n");
    printf("===== \n");
}
int main() {
    printf("===== \n");
    printf("          AUTOMATED TEST SYSTEM\n");
}

```

[^]G Help [^]O Write Out [^]W Where Is [^]K Cut [^]T Execute
[^]X Exit [^]R Read File [^]\ Replace [^]U Paste [^]J Justify [^]/

Output:

```

root@Tejasri:~# nano testing.c
root@Tejasri:~# gcc testing.c -o testing
root@Tejasri:~# ./testing
=====
          AUTOMATED TEST SYSTEM
=====

===== ADDITION TESTS =====
Test Case 1: PASS
Expected: 5 | Actual: 5
Test Case 2: PASS
Expected: 30 | Actual: 30
Test Case 3: PASS
Expected: 0 | Actual: 0
Test Case 4: PASS
Expected: 15 | Actual: 15
Passed: 4/4

===== MULTIPLICATION TESTS =====
Test Case 1: PASS
Expected: 6 | Actual: 6
Test Case 2: PASS
Expected: 25 | Actual: 25
Test Case 3: PASS
Expected: -8 | Actual: -8
Test Case 4: PASS
Expected: 0 | Actual: 0
Passed: 4/4

=====
          TEST REPORT
=====
Addition Tests      : Completed
Multiplication Tests : Completed
Output Verification : Completed
Result Comparison   : Completed
Test Automation     : Completed
=====

Testing completed successfully.
root@Tejasri:~#

```